

Phase 4 — Extracted Services Reference

Project: D&D 3.5e Prototype (Unity)

Extraction period: Phase 4A-4J

Total refactored call sites: 400+

Overview

Phase 4 extracted nine focused services from the monolithic `GameManager` partial class. Each service is a **static utility** (or lightweight `MonoBehaviour` in the case of `DispelMagicService`) that owns a single responsibility. All live in the global namespace — no `using` imports are needed between project files.

#	Service	Location	Type	Sites
4A	SpellUtilities	Magic/SpellUtilities.cs	static	32
4B	SpellCastingHelper	Magic/SpellCastingHelper.cs	static	125
4C	TeamUtility	Combat/TeamUtility.cs	static	88
4D	DispelMagicService	Services/DispelMagicService.cs	MonoBehaviour	30
4E	ConcentrationService	Services/ConcentrationService.cs	static	28
4F-H	CombatLogHelper	Combat/CombatLogHelper.cs	static	51+
4I	SpellTargetingService	Services/SpellTargetingService.cs	static	40+
4J	CombatCalculationService	Combat/CombatCalculationService.cs	static	30+

1. SpellUtilities (Assets/Scripts/Spell/Casting/SpellUtilities.cs)

Centralises spell save-DC computation and caster-ability-modifier lookups.

Key Methods

Method	Description
<code>GetSpellSaveDC(caster, spell)</code>	Full DC calc with feats/focus
<code>GetSpellSaveDC(spellLevel, abilityMod)</code>	Raw $10 + \text{level} + \text{mod}$
<code>GetCastingAbilityModifier(stats)</code>	INT for Wizards, WIS for Clerics, CHA for Sorcerers/Bards
<code>IsImmuneToMindAffecting(target)</code>	Checks undead, vermin, construct, ooze
<code>IsImmuneToSleepEffects(target)</code>	Elf racial + type immunities
<code>IsLivingCreatureForFear(target)</code>	Excludes undead/constructs
<code>IsFearSpell(spell)</code>	Checks Fear descriptor

2. SpellCastingHelper (Assets/Scripts/Spell/Casting/SpellCastingHelper.cs)

Provides caster-level calculations, duration formulas, damage-dice scaling, and the `SpellCastContext` struct for one-stop context building.

Key Methods

Method	Description
<code>BuildContext(caster, spell)</code>	Returns <code>SpellCastContext</code> with CL, DC, name, duration
<code>GetEffectiveCasterLevel(caster, spell)</code>	CL with feats/items/specialisation
<code>GetBaseCasterLevel(stats)</code>	Raw class-level lookup
<code>CalculateDuration(spell, CL)</code>	Rounds from <code>DurationPerLevel × CL</code>
<code>GetDamageDiceCount(CL, maxDice)</code>	<code>min(CL, maxDice)</code>
<code>RollSpellDamage(CL, max, dieSize, label)</code>	Rolls Nd(die) with <code>DiceService</code>
<code>IsBlockedBySpellResistance(target, CL)</code>	Full SR penetration check
<code>PenetratesSpellResistance(caster, target)</code>	Convenience wrapper

SpellCastContext Struct

```
public struct SpellCastContext {
    int CasterLevel, SaveDC;
    string CasterName;
    int DurationRounds => CalculateDuration(...);
    int DamageDice(int maxDice) => GetDamageDiceCount(...);
}
```

3. TeamUtility (Assets/Scripts/Combat/Core/TeamUtility.cs)

Replaces `GameManager.IsEnemyTeam` / `IsAllyTeam` with pure-static queries.

Key Methods

Method	Description
<code>IsEnemy(source, target)</code>	Different team + both alive
<code>IsAlly(source, target)</code>	Same team
<code>IsHumanoid(target)</code>	Type == Humanoid
<code>GetHitDice(target)</code>	Character level / monster HD
<code>GetAliveTeamMembers(teamId, allChars)</code>	Filters alive by team
<code>GetClosestEnemy(source, allChars)</code>	Nearest hostile by distance

4. DispelMagicService (Assets/Scripts/Services/DispelMagicService.cs)

Full dispel/counterspell engine — the only non-static service (needs scene callbacks for UI, character lists, and effect cleanup).

Key Methods

Method	Description
<code>Initialize(...)</code>	Injects <code>Func<CombatUI></code> , character providers, cleanup hooks
<code>PerformDispelCheck(CL, targetCL, isOwn)</code>	$d20 + CL \geq 11 + targetCL$
<code>RollDispelCheck(CL)</code>	Raw $d20 + \min(CL, cap)$
<code>GetDispelDC(targetCL)</code>	$11 + targetCL$
<code>PerformTargetedDispel(caster, target)</code>	Strips highest-CL buff
<code>PerformAreaDispel(caster, targets)</code>	AoE: one check per creature
<code>ExpireReadiedCounterspell(char)</code>	Cleanup on turn end

5. ConcentrationService (Assets/Scripts/Services/ConcentrationService.cs)

All concentration-check DC formulas and success-chance math.

Constants

```
DEFENSIVE_CASTING_DC_BASE = 15    DAMAGE_DC_BASE = 10
GRAPPLED_DC_BASE         = 20    VIGOROUS_MOTION_DC_BASE = 10
VIOLENT_MOTION_DC_BASE   = 15    ENTANGLED_DC_BASE = 15
```

Key Methods

Method	Description
<code>GetDefensiveCastingDC(spellLvl)</code>	<code>15 + spellLevel</code>
<code>GetDamageDC(dmg, spellLvl)</code>	<code>10 + damage + spellLevel</code>
<code>GetGrappledCastingDC(spellLvl)</code>	<code>20 + spellLevel</code>
<code>CalculateSuccessChancePercent(bonus, dc)</code>	<code>5 × (21 - (dc - bonus))</code> clamped
<code>GetConcentrationBonus(caster)</code>	Skill rank + CON + Combat Casting
<code>IsConcentratingOnSpell(caster, id)</code>	Checks active concentration effect

6. CombatLogHelper (Assets/Scripts/Combat/Logging/CombatLogHelper.cs)

25 named colour constants + semantic formatting helpers. Returns Unity rich-text strings — callers still push via `CombatUI?.ShowCombatLog(...)`.

Colour Constants (selected)

Constant	Hex	Usage
<code>ColorGold</code>	<code>#FFD700</code>	Notable / special
<code>ColorGray</code>	<code>#AAAAAA</code>	Neutral info
<code>ColorBrightRed</code>	<code>#FF8888</code>	Damage
<code>ColorGreen</code>	<code>#88FF88</code>	Success / heals
<code>ColorCyan</code>	<code>#88FFEE</code>	Spell cast / buff
<code>ColorRed</code>	<code>#FF6666</code>	Failures
<code>ColorDarkRed</code>	<code>#FF4444</code>	Critical failures

Key Methods

Method	Returns
<code>Color(text, hex)</code>	<code><color=#hex>text</color></code>
<code>Damage(emoji, msg)</code>	Bright-red damage line
<code>Success(emoji, msg)</code>	Green success line
<code>Failure(emoji, msg)</code>	Red failure line
<code>Info(emoji, msg)</code>	Gray informational line
<code>SaveResult(target, success, type, roll, dc)</code>	Green/red save line
<code>ConditionFaded(emoji, name, effect)</code>	Gray “X’s Y fades.”
<code>SpellResisted(target, spell)</code>	Green resist line

7. SpellTargetingService (Assets/Scripts/Services/SpellTargetingService.cs)

Creature-type detection, alignment queries, HD filters, and composite validators used by 20+ spell implementations.

Key Methods

Method	Description
<code>IsValidAliveTarget(target)</code>	Not null, not dead
<code>PassesBasicTargetingChecks(caster, target, spell)</code>	Alive + range + LOS
<code>IsHumanoid / IsUndead / IsConstruct / IsAnimal / IsPlant</code>	Type checks
<code>IsOutsiderOrExtrplanar(target)</code>	Outsider type or extraplanar flag
<code>IsLivingCreature(target)</code>	Not undead/construct
<code>GetCreatureType(target)</code>	String type label
<code>IsWithinHDLimit(target, max)</code>	HD $\leq$ cap
<code>IsValidPersonSpellTarget(target)</code>	Humanoid + $\leq$ medium
<code>IsValidHumanoidMindAffectingTarget(target)</code>	Humanoid + not immune
<code>FilterByCreatureType(list, type)</code>	List filter
<code>IsGoodAligned / IsEvilAligned(target)</code>	Alignment axis check

8. CombatCalculationService (Assets/Scripts/Combat/Utilities/CombatCalculationService.cs)

Pure-math combat formulas — hit determination, AC modifiers, STR scaling, critical-threat math, concealment, and opposed checks.

Constants

```

FightingDefensivelyACBonus      = 2
FightingDefensivelyAttackPenalty = -4
PinnedACPenalty                 = -4
ConcealmentMissChance           = 20
TotalConcealmentMissChance      = 50
BlindedAttackerMissChance       = 50

```

Key Methods

Method	Description
<code>IsHit(nat, total, ac)</code>	Nat-1 auto-miss, nat-20 auto-hit, else total $\geq$ AC
<code>SimpleTouchAC(stats)</code>	10 + DEX + size
<code>ProneACModifier(ranged)</code>	+4 ranged / -4 melee
<code>TwoHandedStrDamage(str)</code>	$\text{floor}(\text{str} \times 1.5)$
<code>OffHandStrDamage(str)</code>	$\text{floor}(\text{str} \times 0.5)$
<code>ClampMinimumDamage(raw)</code>	$\text{max}(1, \text{raw})$
<code>IsCriticalThreat(nat, min)</code>	$\text{nat} \geq \text{min}$
<code>DoubledThreatMin(base)</code>	$21 - 2 \times (21 - \text{base})$
<code>CritBonusDice(dice, mult)</code>	$\text{dice} \times (\text{mult} - 1)$
<code>ConcealmentMiss(roll, pct)</code>	$\text{roll} \leq \text{pct}$
<code>OpposedCheckWins(atk, def)</code>	$\text{atk} \geq \text{def}$

Testing

Test templates live in `Assets/Scripts/Tests/Services/`. Run all via:

```
Tests.Services.ServiceTestRunner.RunAll();
```

Each test class follows the project's existing pattern: a static `RunAll()` method with `Assert(condition, name)` helpers, logging to `Debug.Log`.

Migration Guide

To convert an existing `ShowCombatLog` call:

```
// Before
CombatUI?.ShowCombatLog($"<color=#AAAAAA>♥ {name}'s Shield fades.</color>");

// After
CombatUI?.ShowCombatLog(CombatLogHelper.ConditionFaded("♥", name, "Shield"));
```

To use combat math:


```
// Before
if (roll == 20 || (roll != 1 && total >= targetAC)) { /* hit */ }

// After
if (CombatCalculationService.IsHit(roll, total, targetAC)) { /* hit */ }
```